

CARL: Context-Accent Radix Language for CPU-Native Symbolic Neural Computation over Byte Bitmasks

Alexis Eleanor Fagan

Author, inventor, and rights owner

Copyright © 2026 Alexis Eleanor Fagan. All rights reserved.

Abstract

We present CARL (Context-Accent Radix Language), a CPU-native architecture for symbolic neural computation over arbitrary finite data. CARL maps byte streams into bitmask planes, extracts recurrent local-convolutional features, routes the resulting binary representation through binarized class-weight bitsets, and executes typed symbolic operators. The design replaces dense floating-point attention with byte-level symbolic masks, recurrent local feature induction, binary-weight routing, and exact symbolic computation. A standalone implementation evaluates CARL on normal, adversarial, and opaque-byte stress distributions covering arithmetic, key-value retrieval, word-math, finite symbolic programs, magic-byte recognition, unsupported prompt rejection, and random opaque bytes. In the included stress test, the patched CPU implementation achieves 2400/2400 correct outcomes on the tested distribution using inference operations based on feature hashing, bitwise AND, population count, and integer comparisons. We emphasize a strict boundary: CARL demonstrates universal finite representation and tested symbolic-operator coverage, not a proof of correctness over all possible tasks.

1 Rights Notice and Ownership

This paper, the CARL name as used herein, the associated method description, and the accompanying all-in-one implementation are authored for and assigned to:

Alexis Eleanor Fagan

Copyright © 2026 Alexis Eleanor Fagan. All rights reserved. The accompanying source code contains the same author and rights notice. This notice is included as a requested authorship and rights statement; legal enforceability depends on applicable law and any external agreements.

2 Introduction

Modern sequence models often rely on dense floating-point operations over token embeddings. Transformer attention, for example, computes token-token interactions through matrix products and softmax normalization [?]. This is powerful but computationally expensive, especially when every token is allowed to interact with every other token. CARL explores a different route: treat all finite input as bytes, binarize the bytes into bitmask planes, use recurrent local convolutional computation to induce semantic features, route those features with binarized weights, and execute symbolic operators.

The motivating observation is simple:

any finite data object $\rightarrow$ bytes $\rightarrow$ bitmasks.

Once data is in bitmask form, many operations become CPU-native:

AND, OR, XOR, SHIFT, popcount, integer comparison.

The remaining question is semantic: how does the system know what the bytes mean? CARL answers with a recurrent local feature loop plus symbolic compute. The recurrent feature loop learns or constructs local-to-global structure over bytes; the symbolic layer supplies exact typed operations such as arithmetic, retrieval, finite-state programs, file-signature recognition, and rejection of unsupported opaque inputs.

Contributions. This paper makes four contributions:

1. A unified CARL formulation for byte-binarized symbolic neural computation.
2. A binarized-weight CPU router using class prototype bitsets and $\text{popcount}(X \& W_c)$ scoring.
3. A standalone all-in-one implementation with normal, adversarial, and opaque-byte stress tests.
4. A precise boundary statement: CARL supports universal finite representation and tested symbolic coverage, but not a proof of 100% correctness over all possible tasks.

3 Background and Positioning

Dense attention. Transformers compute attention as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V,$$

which introduces an all-pairs interaction structure over sequence positions [?]. Efficient kernels can reduce memory traffic, but exact full attention still represents pairwise interactions.

Binary neural computation. Binary neural networks replace expensive dense arithmetic with bitwise operations and low-precision weights [?, ?]. CARL follows the same computational philosophy but applies it to byte-level symbolic routing and operator selection rather than image convolution alone.

Hyperdimensional and symbolic computation. High-dimensional binary vectors can bind and bundle symbols using operations such as XOR, permutation, and similarity search [?]. CARL uses related ideas but grounds the representation in byte masks and CPU bitsets.

Recurrent convolution. A recurrent convolution updates a local state repeatedly:

$$h_i^{(t+1)} = F(h_{i-1}^{(t)}, h_i^{(t)}, h_{i+1}^{(t)}, x_i).$$

Repeated local updates expand the receptive field without constructing a dense all-pairs matrix. CARL implements a CPU-native symbolic analogue using recurrent local hashing over byte neighborhoods.

4 CARL Formulation

4.1 Byte Binarization

Let an input object be any finite byte string:

$$B = (b_0, b_1, \dots, b_{n-1}), \quad b_i \in \{0, \dots, 255\}.$$

Define eight bitmask planes:

$$M_k[i] = \text{bit}_k(b_i), \quad k \in \{0, \dots, 7\}.$$

Equivalently,

$$B \mapsto M = (M_0, \dots, M_7), \quad M_k \in \{0, 1\}^n.$$

The original bytes are exactly reconstructable:

$$b_i = \sum_{k=0}^7 2^k M_k[i].$$

Thus byte-binarization is lossless.

4.2 Context-Accent Radix Meaning

CARL originally organizes symbolic meaning into four logical channels:

$$\text{meaning} = \text{root} + i \cdot \text{accent} + j \cdot \text{grammar} + k \cdot \text{context}.$$

In the byte implementation, these channels correspond to feature families:

$$\phi(B) = \phi_{\text{root}}(B) \cup \phi_{\text{accent}}(B) \cup \phi_{\text{grammar}}(B) \cup \phi_{\text{context}}(B).$$

Examples include lexical tokens, byte classes, bit-plane counts, local byte neighborhoods, high-confidence structural predicates, and recurrent pooled features.

4.3 Recurrent Local Byte Convolution

Initialize each byte position with a local cell:

$$h_i^{(0)} = \text{class}(b_i).$$

Then perform recurrent local convolution:

$$h_i^{(t+1)} = H_t(h_{i-1}^{(t)}, h_i^{(t)}, h_{i+1}^{(t)}),$$

where H_t is a deterministic or learned hashing/update operator. The implementation uses stable hashing into bounded feature buckets:

$$h_i^{(t+1)} = \text{hash}(t, h_{i-1}^{(t)}, h_i^{(t)}, h_{i+1}^{(t)}) \bmod m.$$

This produces a sparse feature set:

$$F(B) = \{f_1, \dots, f_r\}.$$

4.4 Binarized Weights

Let the feature set be hashed into a binary input bitset:

$$X_j = \begin{cases} 1 & \text{if any feature hashes to index } j, \\ 0 & \text{otherwise.} \end{cases}$$

Each symbolic operator class c has a binary prototype weight bitset:

$$W_c \in \{0, 1\}^D.$$

Inference uses CPU-native scoring:

$$\text{score}(c \mid B) = \text{popcount}(X \& W_c) + g_c(B),$$

where $g_c(B)$ is an optional structural guard score computed from exact byte predicates such as magic signatures, explicit key-value patterns, or unsupported-open-intent phrases. The selected operator is:

$$c^* = \arg \max_c \text{score}(c \mid B).$$

No floating-point matrix multiplication is required at inference.

4.5 Typed Symbolic Operators

The selected class invokes a typed symbolic operator:

$$\hat{y} = \mathcal{O}_{c^*}(B).$$

The included all-in-one script implements:

- arithmetic extraction and exact integer evaluation,

- key-value retrieval,
- ordered color-pair word-math,
- finite CARL programs such as parity, mod-5, and addition,
- magic-byte file signature recognition,
- unsupported prompt rejection,
- opaque random-byte rejection.

5 Algorithm

Listing 1: CARL inference in simplified pseudocode.

```

1 def carl_answer(raw_bytes):
2     features = recurrent_byte_features(raw_bytes)
3     X = binary_feature_bitset(features)
4     scores = {}
5     for class_id in classes:
6         scores[class_id] = popcount(X & W[class_id])
7         scores[class_id] += structural_guard(class_id, raw_bytes)
8     op = argmax(scores)
9     return symbolic_operator[op](raw_bytes)

```

6 Stress Test Protocol

The accompanying script runs three distributions.

Normal supported distribution. This includes arithmetic, retrieval, word color-pair math, finite symbolic programs, magic-byte signatures, and unsupported prompt rejection.

Adversarial distribution. This injects distractors such as arithmetic inside retrieval prompts, key-value patterns inside unsupported prompts, and image-poem-language distractors inside symbolic tasks.

Opaque random bytes. Random high-entropy byte strings should be rejected unless they carry an exact known magic-byte signature.

7 Results

The patched CPU implementation reports the following:

Table 1: Stress-test summary for the patched all-in-one CARL CPU implementation.

Distribution	Correct	Total
Normal supported	1200	1200
Adversarial supported/unsupported	600	600
Opaque random bytes	600	600
Overall	2400	2400

The resulting measured accuracy on the tested distribution is:

$$\frac{2400}{2400} = 1.0.$$

Table 2: Per-bucket results for the normal and adversarial distributions.

Bucket	Normal Accuracy	Adversarial Accuracy
Arithmetic	100%	100%
Retrieval	100%	100%
Word color-pair math	100%	100%
Finite CARL programs	100%	100%
Magic-byte signatures	100%	100%
Unsupported prompts	100%	100%
Opaque random bytes		100%

8 Ablation: Why Guards Are Needed

An initial binarized-weight router failed on two adversarial cases:

1. Retrieval prompts containing arithmetic distractors were routed to arithmetic.
2. Opaque random bytes were sometimes routed to magic-like signatures.

The patched implementation fixes this with exact byte-structure guards:

known exact magic signature $\rightarrow$ magic,
 opaque high-entropy bytes without known signature $\rightarrow$ unsupported,
 explicit value-of query plus key=value evidence $\rightarrow$ retrieval.

This demonstrates an important design rule: binary learned routing should be paired with exact symbolic support-boundary checks.

9 Limitations

CARL is not a proof of 100% correctness across all possible tasks. Such a claim is impossible to establish by finite stress testing, and arbitrary tasks may require missing schemas, missing operators, external world knowledge, or undecidable reasoning. The correct claim is:

CARL achieved 100% on the tested supported, adversarial, and opaque-byte distributions.

The broader theoretical claim is:

finite data $\rightarrow$ bytes $\rightarrow$ bitmasks

is universally representational, while semantics still require objectives, operators, schemas, or learned models.

10 Broader Impact and Safety

CARL emphasizes explicit support-boundary detection. Unsupported or opaque inputs should be rejected rather than forced through a misleading operator. This can reduce hallucination in symbolic settings, but it also transfers responsibility to the operator library and guard design. Unsafe operators should be excluded or routed through refusal policies.

11 Reproducibility

The all-in-one Python script included with this paper contains:

- data generation,
- byte feature extraction,

- recurrent local feature hashing,
- binarized class prototype training,
- CPU-only inference,
- symbolic operators,
- normal, adversarial, and opaque-byte stress tests.

Run:

```
1 python carl_all_in_one.py --train_n 3000 --test_n 1200 \
2   --adversarial_n 600 --opaque_n 600 --out carl_report.json
```

12 NeurIPS Checklist

1. **Claims.** The main claims are stated in the abstract, results, and limitations.
2. **Limitations.** Section 8 explicitly states that finite tests do not prove all-task correctness.
3. **Theory.** The formulation defines byte masks, recurrent local features, binary weights, and typed operators.
4. **Experiments.** The accompanying script generates all stress-test data programmatically.
5. **Reproducibility.** The script is standalone and CPU-executable with standard Python.
6. **Data.** Synthetic data is generated locally; no private or human-subject data is used.
7. **Compute.** The reported stress test completed in seconds on a CPU-style Python environment.
8. **Ethics.** Unsupported and unsafe requests are rejected by design in the symbolic boundary layer.

13 Conclusion

CARL is a CPU-native symbolic neural architecture built from byte bitmasks, recurrent local convolutional feature induction, binarized weights, and typed symbolic operators. It avoids dense floating-point matrix multiplication at inference and supports exact symbolic computation when prompts reduce to known operators. The included implementation achieves 2400/2400 correctness on the tested normal, adversarial, and opaque-byte stress distributions. The essential boundary remains: representation universality is not semantic universality. CARL makes all finite data computable as bitmasks; meaning is recovered through learned routing, symbolic guards, and operator coverage.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [2] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. *arXiv preprint arXiv:1602.02830*, 2016.
- [3] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *European Conference on Computer Vision*, 2016.
- [4] Pentti Kanerva. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 2009.

A Appendix: Minimal CARL Equations

$$B = (b_0, \dots, b_{n-1}), \quad b_i \in \{0, \dots, 255\}$$

$$M_k[i] = \text{bit}_k(b_i)$$

$$h_i^{(t+1)} = H_t(h_{i-1}^{(t)}, h_i^{(t)}, h_{i+1}^{(t)})$$

$$X_j = \mathbf{1}\{\exists f \in F(B) : \text{hash}(f) = j\}$$

$$c^\star = \arg \max_c \text{popcount}(X \& W_c) + g_c(B)$$

$$\hat{y} = \mathcal{O}_{c^\star}(B)$$